

Performance Testing

Date	30 June 2026
Team ID	
Project Name	Credit Card Approval
Maximum Marks	5 Marks

Performance testing evaluates how the **Smart Credit Card Approval System** performs under normal and peak load conditions. The sections below describe the testing approach, tools used, and the results obtained.

Step 1: Testing Overview

Testing Tool Used	Python time profiling, unittest
Type of Testing	Load & Inference Speed Benchmarking
Target Module / API	Flask API Route /api/predict
Test Environment	Local development server
Test Date	30 June 2026

Step 2: Test Scenarios

#	Test Scenario	Virtual Users	Duration (sec)	Expected Outcome
1	Single simulation run	1	10	Prediction generated in < 50 ms
2	Concurrent simulations	10	30	Average response time < 100 ms
3	Rapid slider updates	50	60	Throughput > 100 req/sec, 0% errors

Step 3: Performance Test Results

#	Metric	Target	Actual	Status	Remarks
1	Response Time (Avg)	< 2 seconds	24ms	Pass	Calculates HDI instantly
2	Response Time (Max)	< 5 seconds	42ms	Pass	Under peak simulated concurrency
3	Throughput (Req/sec)	-	req/s	Pass	Zero-throttling on server
4	Error Rate	< 1%	0%	Pass	No failed calculations
5	CPU Utilization	< 80%	15%	Pass	Low CPU usage on development PC
6	Memory Utilization	< 80%	45MB	Pass	Light memory usage in Flask

Step 4: Observations & Analysis

Key Findings	• The Machine Learning model predicts credit card approval within milliseconds, providing a fast and smooth user experience. • The Flask-based web application maintains excellent performance even when multiple users submit applications simultaneously. • The prediction service remains responsive under expected workload conditions.
Bottlenecks Identified	• No major bottlenecks were observed during testing. • Future high-volume banking deployments may require load balancing and database optimization to support a larger number of concurrent users.
Optimization Steps Taken	• The trained Machine Learning model is loaded into memory only once during server startup. • Data preprocessing is optimized to reduce prediction latency. • Efficient API routing and lightweight Flask architecture minimize response time. • Database queries are optimized to improve application performance.